

From Pharo-Side State to a GemStone Server-Side Website

From Pharo-Side State to a GemStone Server-Side Website

How an OkZulu audit turned a working Seaside application into a more durable, operationally safer GemStone-backed system.

The Starting Point

OkZulu started as a familiar kind of Smalltalk web application: a Pharo Seaside image held most of the application shape, rendered the website, and coordinated persistence through a bridge into GemStone.

That worked. It was productive. It made feature development fast. But as the application grew into payments, ticketing, fundraising, organiser dashboards, refunds, webhooks, email delivery, audits, and operational reporting, the original split became harder to defend.

The Pharo image was doing too much.

It was not just rendering pages. It also held large in-memory repository structures, performed business workflows, mirrored state into GemStone, retried failed writes, detected drift, and maintained fallback paths for older script-based operations. GemStone was present, but it was not yet the clear source of truth for all important state transitions.

The audit question was simple:

Are we really using GemStone as the server-side application database, or are we still treating it as a remote persistence target behind a Pharo-side application store?

The first answer was uncomfortable: kind of.

The bridge was valuable, but the architecture still had too much dual-store behaviour. Pharo and GemStone could diverge. If a Pharo-side update succeeded and a GemStone upsert failed, recovery depended on mirror helpers, write-ahead logs, drift detectors, and repair jobs. Those tools were useful during migration, but they were also evidence that the system was not yet clean.

The Core Problem: Dual Ownership

The original repository model had two competing responsibilities:

- Pharo held application collections and feature stores locally.
- GemStone held durable canonical state.
- Services often wrote to both.
- The bridge included fallback paths that could execute older GemStone scripts when resident repositories were not available.
- Operational tools were needed to detect and repair divergence.

That design made sense as an incremental migration path. It avoided a big-bang rewrite. But it had a cost: every important workflow needed to answer the same question twice.

Did the local repository update?

Did the GemStone write also succeed?

If the answers differed, the system had to recover. The more features OkZulu added, the more expensive that model became.

Payments made the problem more serious. A ticket purchase, refund approval, payment webhook, sponsorship donation, or organiser notification is not just UI state. It is business state. It needs durable ordering, idempotency, clear ownership, and auditable outcomes.

The goal of the transition was therefore not “move code for neatness.” The goal was to remove ambiguity about where truth lives.

The Target Architecture

The target architecture was:

- Pharo Seaside remains the web presentation layer.
- GemStone owns durable domain state and server-side repositories.
- Pharo-side repositories become thin caches or dispatchers, not parallel stores.
- Business mutations happen through GemStone resident repository operations.
- Bridge calls send remote messages, not dynamically generated scripts.
- Fallback execution is retired in production.
- Operational checks prove that the old world is gone, not merely hidden.

In practical terms, that meant turning GemStone from “a database we write to” into “the server-side object environment that owns the domain.”

That is the important distinction.

A database can store records. A GemStone server can store objects, indexes, repository classes, and business operations close to the data. For a Smalltalk system, this is a major advantage: the same conceptual model can live on the durable server side instead of being flattened into a passive data store behind the web image.

Phase 1: Stop Building GemStone Code as Strings

One of the first audit findings was script construction.

The old bridge had code paths that built GemStone Smalltalk source strings from Pharo values and then evaluated those scripts remotely. That pattern is fragile. It creates quoting risks. It makes source auditing difficult. It also blurs the difference between sending data and generating code.

The improved bridge moved away from script builders and toward resident classes plus remote message sends.

The audit gate was deliberately blunt:

- No `evaluateScript` paths in the gateway for normal domain operations.
- No string escaping helpers such as `quoteString:` used as part of the persistence path.
- No identifier validation helper needed for dynamic script construction.
- Resident repository classes deployed into GemStone and called directly.

This was not just a security cleanup. It changed the operational shape of the system. When Pharo sends `checkoutBookingId:paymentId:` or an equivalent repository message to GemStone, the server owns the transaction boundary. The bridge becomes a dispatcher rather than a code generator.

That is easier to test, easier to audit, and easier to reason about under failure.

Phase 2: Introduce GemStone Resident Repositories

The next step was to create server-side repository classes inside GemStone.

Examples include repositories for bookings, events, users, payment records, comments, organiser groups, reviews, media assets, promo codes, and state-store records. These classes own the canonical collections and indexes on the GemStone side.

The important change was not just adding classes. It was moving behaviour.

Instead of Pharo reading a local collection, filtering it, mutating it, and then trying to mirror the result, GemStone can answer indexed reads and perform mutations directly. Server-side operations can update objects and indexes in one durable transaction.

This matters for workflows such as:

- Creating a booking.

- Confirming checkout.
- Marking payment records paid.
- Cancelling or refunding a ticket.
- Updating event and attendee indexes.
- Recording operational evidence.
- Reading feature-store records for dashboards.

Once these operations live server-side, the Pharo image no longer needs to be trusted as a durable state holder.

How the Code Migration Actually Worked

The migration was deliberately mechanical. The safest way to move a running Pharo website to GemStone server-side ownership was not to rewrite the UI first. The UI and service selectors stayed mostly stable while the implementation underneath moved from local collections and script builders to resident GemStone repositories.

The pattern was:

- Keep the public Pharo service API stable.
- Move canonical storage and indexes into GemStone resident classes.
- Replace local repository reads with GemStone-backed read-through calls.
- Replace local mutations with one remote server-side operation.
- Commit through the bridge with conflict detection.
- Delete fallback code only after the resident path was proven.

Before: Pharo Owned the Collection

The old repository shape was typical for a productive Seaside application. The repository lazily allocated local collections inside the Pharo image.

```
OkzRepository >> bookings
  bookings ifNil: [ bookings := OrderedCollection new ].
  ^ bookings
```

```
OkzRepository >> bookingWithId: aBookingId
  ^ self bookings
    detect: [ :booking | booking id = aBookingId ]
    ifNone: [ nil ]
```

```
OkzRepository >> bookingsForUserId: aUserId
  ^ (self bookings select: [ :booking |
    booking userId = aUserId ]) asOrderedCollection
```

That is fast and simple, but it makes the Pharo image a state holder. If the image restarts, that local collection has to be rebuilt. If a write to GemStone fails after this collection has changed, Pharo and GemStone diverge.

The write path had the same problem.

```
OkzRepository >> upsertBooking: anOkzBooking
  | existing |
  existing := self bookingWithId: anOkzBooking id.
```

```

existing
  ifNil: [ self bookings add: anOkzBooking ]
  ifNotNil: [
    self bookings
      at: (self bookings indexOf: existing)
      put: anOkzBooking ].
self mirrorBookingUpsertInGemStone: anOkzBooking.
^ anOkzBooking

```

The local update happened first. GemStone was updated second. That was the dual-write hazard.

Before: GemStone Scripts Were Built as Strings

Some older bridge paths sent GemStone source code as a string. A simplified version looked like this:

```

OkzGemStoneGateway >> legacyUpsertBooking: anOkzBooking
  | script |
  script := String streamContents: [ :stream |
    stream
      nextPutAll: 'OkzRepository current
upsertBookingFromDictionary: ';
      nextPutAll: (self literalForDictionary: anOkzBooking
asDictionary);
      nextPutAll: '. System commitTransaction.' ].
  ^ self evaluateScript: script

```

This has three problems.

First, user data and executable source are too close together. Every string needs perfect escaping. Second, the gateway becomes a source-code generator instead of a bridge. Third, it is hard to audit because the real server-side operation is assembled dynamically.

The migration replaced this with resident classes and remote messages.

After: GemStone Owns the Repository

The GemStone side gets a real repository class. The exact deployment mechanism can vary, but the class shape is intentionally plain Smalltalk.

```

Object subclass: #OkzBookingRepository
  instanceVariableNames: ''
  classVariableNames: 'BookingsById BookingIdsByUser
BookingIdsByEvent'
  poolDictionaries: ''
  category: 'Okz-Server'

```

The class owns its canonical dictionaries and indexes.

```

OkzBookingRepository class >> initializeRepository
  BookingsById ifNil: [ BookingsById := Dictionary new ].

```

```

    BookingIdsByUser ifNil: [ BookingIdsByUser := Dictionary
new ].
    BookingIdsByEvent ifNil: [ BookingIdsByEvent := Dictionary
new ].

```

```

OkzBookingRepository class >> bookingsById
    self initializeRepository.
    ^ BookingsById

```

```

OkzBookingRepository class >> bookingWithId: aBookingId
    self initializeRepository.
    ^ BookingsById at: aBookingId ifAbsent: [ nil ]

```

The server-side upsert owns both the object and its indexes.

```

OkzBookingRepository class >> upsert: anOkzBooking
    self initializeRepository.
    BookingsById at: anOkzBooking id put: anOkzBooking.
    self indexBooking: anOkzBooking.
    ^ anOkzBooking

```

```

OkzBookingRepository class >> indexBooking: anOkzBooking
    self
        addBookingId: anOkzBooking id
        toIndex: BookingIdsByUser
        key: anOkzBooking userId.
    self
        addBookingId: anOkzBooking id
        toIndex: BookingIdsByEvent
        key: anOkzBooking eventId.

```

```

OkzBookingRepository class >> addBookingId: aBookingId toIndex:
anIndex key: aKey
    | ids |
    (aKey ifNil: [ '' ]) isEmpty ifTrue: [ ^ self ].
    ids := anIndex at: aKey ifAbsentPut: [ OrderedCollection
new ].
    (ids includes: aBookingId) ifFalse: [ ids add: aBookingId ].

```

Reads now use server-side indexes instead of scanning a Pharo collection.

```

OkzBookingRepository class >> bookingsForUserId: aUserId
    | ids |
    self initializeRepository.
    ids := BookingIdsByUser at: aUserId ifAbsent: [ #() ].
    ^ ids collect: [ :bookingId | BookingsById at: bookingId ]

```

```

OkzBookingRepository class >> activeBookingsForEventId: anEventId
    | ids |
    self initializeRepository.
    ids := BookingIdsByEvent at: anEventId ifAbsent: [ #() ].
    ^ (ids collect: [ :bookingId | BookingsById at: bookingId ])
        select: [ :booking | booking status ~= 'cancelled' ]

```

The important change is ownership. Pharo can cache these answers, but GemStone owns the collection and the indexes.

After: The Gateway Sends a Remote Message

The Pharo gateway becomes a thin dispatcher. A representative bridge call looks like this:

```
OkzGemStoneGateway >> upsertBooking: anOkzBooking
  ^ self withAvailableConnectionDo: [ :session |
    | root repository result |
    root := session bridgeRoot.
    repository := root at: #OkzBookingRepository.
    result := repository
      perform: #upsert:
        withArguments: { anOkzBooking }.
    session commitTransactionOrSignalConflict.
    result ]
```

The details depend on the bridge session object, but the shape is the point:

- Resolve a resident GemStone class from the bridge root.
- Send a selector with normal Smalltalk arguments.
- Commit once at the bridge boundary.
- Signal a conflict instead of silently continuing.

No source string is generated. No user text is interpolated into executable GemStone code.

The same shape works for reads.

```
OkzGemStoneGateway >> bookingsForUserId: aUserId
  ^ self withAvailableConnectionDo: [ :session |
    | repository |
    repository := session bridgeRoot at:
#OkzBookingRepository.
    repository
      perform: #bookingsForUserId:
        withArguments: { aUserId } ]
```

The Pharo Repository Becomes a Cache

Once the GemStone repository exists, the Pharo repository should stop allocating authoritative collections.

Before:

```
OkzRepository >> bookingsForUserId: aUserId
  ^ (self bookings select: [ :booking |
    booking userId = aUserId ]) asOrderedCollection
```

After:

```
OkzRepository >> bookingsForUserId: aUserId
  ^ self readThroughCache
    key: 'bookingsForUser:', aUserId
    fetch: [ self gemstoneGateway bookingsForUserId: aUserId ]
    ttlSeconds: 30
```

The cache is allowed to improve page rendering performance. It is not allowed to become the source of truth.

Writes invalidate cache entries after GemStone succeeds.

```
OkzRepository >> upsertBooking: anOkzBooking
  | saved |
  saved := self gemstoneGateway upsertBooking: anOkzBooking.
  self readThroughCache removeKey: 'booking:', anOkzBooking id
ifAbsent: [ ].
  self readThroughCache removeKey: 'bookingsForUser:',
anOkzBooking userId ifAbsent: [ ].
  self readThroughCache removeKey: 'bookingsForEvent:',
anOkzBooking eventId ifAbsent: [ ].
  ^ saved
```

The ordering matters. Cache invalidation happens after the canonical server-side write succeeds.

Moving a Whole Workflow: Checkout

The most valuable functions to move were not simple CRUD methods. They were workflows where multiple objects and indexes must change together.

A checkout confirmation is a good example. In the old Pharo-heavy style, a service could touch a booking, a payment record, an event counter, notifications, and local indexes across several method calls.

```
OkzCheckoutOperationsService >> confirmCheckoutBookingId:
aBookingId paymentId: aPaymentId
  | booking payment |
  booking := repository bookingWithId: aBookingId.
  payment := repository paymentRecordWithId: aPaymentId.
  payment status: 'paid'.
  booking status: 'booked'.
  booking paidAt: DateAndTime now asString.
  repository upsertPaymentRecord: payment.
  repository upsertBooking: booking.
  repository queueTicketIssuedNoticeForBooking: booking.
  ^ booking
```

That looks reasonable, but it spreads the durable transition across Pharo methods. During the migration, the durable part moved to a GemStone server-side operation.

```
OkzPaymentRecordRepository class >> checkoutBookingId: aBookingId
paymentId: aPaymentId
  | booking payment now |
  booking := OkzBookingRepository bookingWithId: aBookingId.
```

```

        booking ifNil: [ self error: 'Booking not found: ',
aBookingId ].
        payment := self paymentRecordWithId: aPaymentId.
        payment ifNil: [ self error: 'Payment not found: ',
aPaymentId ].

        now := DateAndTime now asString.
        payment
            status: 'paid';
            updatedAt: now.
        booking
            status: 'booked';
            paidAt: now;
            paymentRecordId: payment id.

        self upsert: payment.
        OkzBookingRepository upsert: booking.
        OkzEventRepository incrementAttendeeCountForEventId: booking
eventId.

        ^ Dictionary new
            at: #booking put: booking;
            at: #paymentRecord put: payment;
            at: #status put: 'confirmed';
            yourself

```

The Pharo gateway then calls exactly one server-side operation.

```

OkzGemStoneGateway >> checkoutBookingId: aBookingId paymentId:
aPaymentId
    ^ self withAvailableConnectionDo: [ :session |
        | repository result |
        repository := session bridgeRoot at:
#OkzPaymentRecordRepository.
        result := repository
            perform: #checkoutBookingId:paymentId:
                withArguments: { aBookingId. aPaymentId }.
        session commitTransactionOrSignalConflict.
        result ]

```

The Pharo service becomes orchestration around the durable operation, not the owner of the operation.

```

OkzCheckoutOperationsService >> confirmCheckoutBookingId:
aBookingId paymentId: aPaymentId
    | result booking |
    result := repository checkoutBookingId: aBookingId paymentId:
aPaymentId.
    booking := result at: #booking.
    repository queueTicketIssuedNoticeForBooking: booking.
    ^ booking

```

Even here, notification queuing can be moved server-side later if it must be part of the same durable transaction. The migration did not require every method to move at once. It moved the critical state transition first.

Moving Payment Confirmation and Refund Logic

Payment confirmation had the same before/after shape.

Before, Pharo would find a payment record locally, update it, mirror it, then update related booking or sponsorship state.

```
OkzRepository >> markPaymentRecordId: aPaymentId
paidWithExternalPaymentId: anExternalId
| payment booking |
payment := self paymentRecordWithId: aPaymentId.
payment
    status: 'paid';
    externalPaymentId: anExternalId;
    updatedAt: DateAndTime now asString.
self upsertPaymentRecord: payment.

booking := self bookingWithId: payment bookingId.
booking ifNotNil: [
    booking status: 'booked'.
    self upsertBooking: booking ].
^ payment
```

After, the server-side repository owns the state transition.

```
OkzPaymentRecordRepository class >> markPaidPaymentRecordId:
aPaymentId externalPaymentId: anExternalId
| payment booking now |
payment := self paymentRecordWithId: aPaymentId.
payment ifNil: [ self error: 'Payment not found: ',
aPaymentId ].

now := DateAndTime now asString.
payment
    status: 'paid';
    externalPaymentId: anExternalId;
    updatedAt: now.
self upsert: payment.

payment bookingId isEmpty ifFalse: [
    booking := OkzBookingRepository bookingWithId: payment
bookingId.
    booking ifNotNil: [
        booking
            status: 'booked';
            paidAt: now.
        OkzBookingRepository upsert: booking ] ].

^ payment
```

Refunds followed the same principle. The durable decision, refund status, notes, payment record, and booking status belong together on the server side.

```
OkzPaymentRecordRepository class >> recordRefundForPaymentId:
aPaymentId refundId: aRefundId amountMinor: amountMinor note:
aNote
    | payment booking now |
    payment := self paymentRecordWithId: aPaymentId.
    payment ifNil: [ self error: 'Payment not found: ',
aPaymentId ].

    now := DateAndTime now asString.
    payment
        refundStatus: 'refunded';
        refundedAt: now;
        refundExternalId: aRefundId;
        refundAmountMinor: amountMinor;
        notes: aNote.
    self upsert: payment.

    booking := OkzBookingRepository bookingWithId: payment
bookingId.
    booking ifNotNil: [
        booking
            status: 'refunded';
            refundedAt: now.
        OkzBookingRepository upsert: booking ].

    ^ Dictionary new
        at: #paymentRecord put: payment;
        at: #booking put: booking;
        yourself
```

The bridge call stays small.

```
OkzGemStoneGateway >> recordRefundForPaymentId: paymentId
refundId: refundId amountMinor: amountMinor note: note
    ^ self withAvailableConnectionDo: [ :session |
        | repository result |
        repository := session bridgeRoot at:
#OkzPaymentRecordRepository.
        result := repository
            perform:
#recordRefundForPaymentId:refundId:amountMinor:note:
            withArguments: { paymentId. refundId. amountMinor.
note }.
        session commitTransactionOrSignalConflict.
        result ]
```

Moving Feature Stores

Not everything deserved a first-class repository class immediately. OkZulu also had feature stores for operational records, dashboards, delivery history, sponsorship analytics, migration evidence, and other append-only or dictionary-shaped data.

Before, a feature store could be just a local Pharo collection hidden behind a string key.

```
OkzRepository >> operationalAlertRecords
  ^ self featureCollectionNamed: 'operationalAlertRecords'
```

```
OkzRepository >> recordOperationalAlert: aDictionary
  self operationalAlertRecords addFirst: aDictionary.
  self mirrorFeatureStoreNamed: 'operationalAlertRecords'
```

The first improvement was explicit classification. A durable feature store must appear in the durable list.

```
OkzRepositoryStorageBase >> criticalFeatureStoreNames
  ^ #(
    'paymentLedgerEntries'
    'operationalAlertRecords'
    'webhookIdempotencyLog'
    'refundOperationRecords'
    'fundraisingAnalyticsEvents'
    'fundraisingReviewRecords'
  ) asOrderedCollection
```

Dictionary-shaped stores are classified separately.

```
OkzRepositoryStorageBase >> nativeDictionaryFeatureStoreNames
  ^ #(
    'idempotencyRecordsByKey'
    'runtimeSafetyControlsByKey'
    'sponsorshipCampaignsByEventId'
    'standaloneFundraisingCampaignsById'
  ) asOrderedCollection
```

The state-store repository on the GemStone side then owns generic feature records.

```
OkzStateStoreRepository class >> upsertFeatureRecordsNamed:
featureName records: records
  self initializeRepository.
  FeatureRecordsByName at: featureName put: records
asOrderedCollection.
  ^ records
```

```
OkzStateStoreRepository class >> featureRecordsNamed: featureName
  self initializeRepository.
  ^ FeatureRecordsByName
    at: featureName
    ifAbsent: [ OrderedCollection new ]
```

The bridge sends a state-store message instead of evaluating a script.

```
OkzGemStoneGateway >> upsertFeatureRecordsNamed: featureName
records: records
  ^ self withAvailableConnectionDo: [ :session |
    | repository result |
    repository := session bridgeRoot at:
#OkzStateStoreRepository.
    result := repository
      perform: #upsertFeatureRecordsNamed:records:
        withArguments: { featureName. records }.
    session commitTransactionOrSignalConflict.
    result ]
```

The regression test prevents accidental local-only stores.

```
OkzRepositoryTest >>
testLiteralFeatureStoreAccessorsAreClassifiedForGemStoneMirroring
  | repository referenced unmirrored |
  repository := OkzRepository new.
  referenced := repository
featureStoreNamesReferencedByLiteralAccessors.
  unmirrored := repository unmirroredFeatureStoreNames.

  self assert: (referenced includes: 'supportTickets').
  self assert: (referenced includes:
'draftAutosavesByEventAndUser').
  self
    assert: unmirrored isEmpty
    description: 'Unmirrored feature stores: ', unmirrored
asArray printString
```

That test is intentionally boring. It scans for literal featureCollectionNamed: and featureDictionaryNamed: usage. If a developer adds a new durable store and forgets to classify it, the suite fails.

Empty Remote State Is Not Failure

One bug class in migrations like this is treating an empty remote result as “do nothing.” That leaves stale local state in place.

The corrected rule is:

- nil means unavailable or invalid remote state.
- An empty collection means authoritative empty state.

Before:

```
OkzRepository >> hydrateSupportTicketsFromGemStone
  | remoteRows |
  remoteRows := gateway featureRecordRecordsNamed:
'supportTickets'.
```

```
remoteRows isEmpty ifFalse: [
    self replaceSupportTicketsWith: remoteRows ]
```

After:

```
OkzRepository >> hydrateSupportTicketsFromGemStone
    | remoteRows |
    remoteRows := gateway featureRecordRecordsNamed:
'supportTickets'.
    remoteRows ifNil: [
        self error: 'GemStone supportTickets unavailable' ].
    self replaceSupportTicketsWith: remoteRows
```

If GemStone says there are zero support tickets, the Pharo cache must become empty.

Removing Runtime Fallbacks

During migration, a read-through cache often had a fallback block.

```
OkzGemStoneReadThroughCache >> readKey: key fetchBlock: fetchBlock
localFallbackBlock: fallbackBlock
    | remote |
    remote := fetchBlock on: Error do: [ nil ].
    remote ifNil: [ ^ fallbackBlock value ].
    ^ remote
```

That is useful while proving the remote path. It is wrong after cutover. The production version fails closed.

```
OkzGemStoneReadThroughCache >> readKey: key fetchBlock: fetchBlock
    | remote |
    remote := fetchBlock on: Error do: [ :error |
        self signalUnavailableKey: key reason: error
messageText ].
    remote ifNil: [
        self signalUnavailableKey: key reason: 'GemStone returned
nil' ].
    ^ remote
```

This change turns hidden stale reads into visible operational failures.

Deploying Resident Classes

The migration also needed repeatable deployment of the GemStone classes. A simplified deployment script looks like this:

```
| root |
root := System myUserProfile symbolList objectNamed: #UserGlobals.

(root includesKey: #OkzBookingRepository) ifFalse: [
    Object
        subclass: #OkzBookingRepository
```

```

        instVarNames: #()
        classVars: #(BookingsById BookingIdsByUser
BookingIdsByEvent)
        classInstVars: #()
        poolDictionaries: #()
        inDictionary: root
        options: #() ].

OkzBookingRepository compileMethod: '
initializeRepository
    BookingsById ifNil: [ BookingsById := Dictionary new ].
    BookingIdsByUser ifNil: [ BookingIdsByUser := Dictionary
new ].
    BookingIdsByEvent ifNil: [ BookingIdsByEvent := Dictionary
new ].
' dictionaries: System myUserProfile symbolList.

System commitTransaction

```

In production, the bridge deployment path wraps class creation and method compilation in a conflict-checked transaction. The important idea is that resident classes are deployed as server-side code, not generated ad hoc during each request.

Auditing the Migration

The audit used blunt checks because blunt checks catch architectural backsliding.

```

wc -l src/Okz-Model/OkzGemStoneGateway.class.st
wc -l src/Okz-Model/OkzRepository.class.st
grep -c "evaluateScript\\|quoteString\\|literalForStringOrNil"
src/Okz-Model/OkzGemStoneGateway.class.st
ls src/Okz-Model/OkzWriteAheadLog* 2>/dev/null && echo "WAL still
present"

```

Those checks do not prove correctness alone, but they prove that the old shape is not quietly returning. If the gateway is supposed to be a dispatcher, it should be small. If script builders are retired, the grep count should be zero. If the dual-store model is gone, WAL and drift-repair helpers should not still be active.

The preflight script made these checks part of the release path.

```
./scripts/gemstone_phase6_preflight.sh --skip-ansible
```

A typical passing gate checks:

- Gateway size.
- Repository size.
- Script-builder absence.
- Retired fallback absence.
- Backup producer wiring.
- Restore-drill runbook presence.
- Performance baseline scripts.
- Documentation and PDF generation.

The Migration Checklist

For each domain, the practical checklist was:

1. Identify the Pharo repository methods that own durable state.
2. Create or extend the corresponding GemStone resident repository class.
3. Move indexes to GemStone class variables or server-side structures.
4. Move multi-object state transitions into one GemStone class-side operation.
5. Replace Pharo local scans with remote indexed reads.
6. Replace generated scripts with remote message sends.
7. Commit with `commitTransactionOrSignalConflict` at the bridge boundary.
8. Treat `nil` remote state as unavailable and empty collections as authoritative empty.
9. Invalidate Pharo caches after successful GemStone writes.
10. Delete fallback branches after tests and production evidence prove the new path.

That checklist is what turned the migration from a vague architectural goal into a repeatable engineering process.

Debugging Server-Side GemStone Code

Moving behaviour into GemStone changes how debugging works.

The short answer is: yes, use GemStone-Pharo-Bridge as the developer debugging bridge for server-side GemStone code. It is the tool that lets a Pharo image log in to GemStone, execute resident methods, inspect remote objects, and open `GbsRemoteDebugger` when a server-side exception is signalled.

But there is an important distinction:

- In development and staging, the bridge is the interactive debugger.
- In production, the bridge is not the normal live debugger. Production should use logs, object-log records, reproducible scripts, runbooks, and staging reproduction. Attach an interactive debugger to production only for an explicit emergency.

That distinction matters because a GemStone debugging session is a real GemStone session. It can hold locks, keep transactions open, and inspect or mutate persistent objects.

Load the Debugging Stack

For debugging, load the full developer stack rather than only the production overlay.

```
Metacello new
  baseline: 'GemStonePharo';
  repository: 'tone1:///Users/tariq/src/gemtools/GemStone-Pharo-
Bridge/src';
  load: 'Full'.
```

The Full load includes the MagLev-aware test and tool packages, including the remote debugger support. For production-like application runtime work, MagLev is enough. For interactive debugging, use Full.

Then log in through the GemStone launcher or create a session directly.

```
| session |
session := GbsSessionParameters new
    name: 'OkZulu Debug Session';
    gemStoneName: 'gs64stone';
    username: 'DataCurator';
    password: 'swordfish';
    netldiHostOrIp: 'localhost';
    netldiNameOrPort: '50377';
    login.
```

You can also open the GemStone workspace from Pharo:

```
GbsWorkspace open
```

The workspace evaluates code in GemStone through the active bridge session. If the server-side code raises an error, GbsRemoteDebugger is expected to open in Pharo.

Reproduce the Server-Side Selector Directly

The best way to debug migrated code is to reproduce the exact resident repository selector that the website calls.

If the website calls this gateway method:

```
OkzGemStoneGateway >> checkoutBookingId: aBookingId paymentId:
aPaymentId
    ^ self withAvailableConnectionDo: [ :session |
        | repository result |
        repository := session bridgeRoot at:
#OkzPaymentRecordRepository.
        result := repository
            perform: #checkoutBookingId:paymentId:
                withArguments: { aBookingId. aPaymentId }.
        session commitTransactionOrSignalConflict.
        result ]
```

Do not start by clicking through the whole website. Start by reproducing the server-side message from a GemStone workspace or a small Pharo snippet.

```
| session root repository result |
session := GbsSessionParameters currentSession.
root := session bridgeRoot.
repository := root at: #OkzPaymentRecordRepository.

result := repository
    perform: #checkoutBookingId:paymentId:
        withArguments: { 'booking-123'. 'payment-456' }.
```

```
session commitTransactionOrSignalConflict.  
result inspect.
```

That isolates the problem. If this fails, the bug is in the GemStone resident operation or its data. If this succeeds, the problem is probably in the Pharo service, page flow, payment callback, or argument conversion before the gateway call.

Let GemStone Exceptions Open GbsRemoteDebugger

A server-side error should come back to Pharo as a GbsError. The bridge records the GemStone context or exception object OOP on the error. The remote debugger uses that context to show the stack where possible.

A simple test is:

```
1 / 0
```

evaluated in a GemStone workspace. The expected result is that GbsRemoteDebugger opens.

A better test, because it has local variables and nested frames, is:

```
| bookingId paymentId |  
bookingId := 'missing-booking'.  
paymentId := 'payment-456'.  
OkzPaymentRecordRepository  
    checkoutBookingId: bookingId  
    paymentId: paymentId
```

If the method signals:

```
self error: 'Booking not found: ', aBookingId
```

the debugger should show the server-side error, stack frames, and, when available, frame variables such as bookingId and paymentId.

Some simple errors may not produce a fully walkable stack. That is a limitation of the available GemStone context, not necessarily a failure of the bridge. For real application methods with locals and sends, the debugger is usually much more useful.

Use halt Carefully

For development only, a targeted halt can stop inside the resident GemStone method.

```
OkzPaymentRecordRepository class >> checkoutBookingId: aBookingId  
paymentId: aPaymentId  
    | booking payment |  
    self halt. "Development only. Never leave this in committed  
production code."  
    booking := OkzBookingRepository bookingWithId: aBookingId.
```

```

payment := self paymentRecordWithId: aPaymentId.
...

```

Then run the exact remote message:

```

| repository |
repository := GbsSessionParameters currentSession bridgeRoot
  at: #OkzPaymentRecordRepository.
repository
  perform: #checkoutBookingId:paymentId:
    withArguments: { 'booking-123'. 'payment-456' }

```

The debugger opens at the halt. Inspect the receiver, temporary variables, and arguments. Remove the halt before committing.

Inspect Remote Objects Through Proxies

When GemStone returns a persistent object to Pharo, the bridge may return a proxy. Inspecting the proxy is often enough to answer basic questions.

```

| session bookingRepository booking |
session := GbsSessionParameters currentSession.
bookingRepository := session bridgeRoot at: #OkzBookingRepository.
booking := bookingRepository
  perform: #bookingWithId:
    withArguments: { 'booking-123' }.
booking inspect.

```

For collections, prefer small targeted reads. Do not casually inspect enormous production dictionaries.

```

| repository bookings |
repository := GbsSessionParameters currentSession bridgeRoot
  at: #OkzBookingRepository.
bookings := repository
  perform: #bookingsForUserId:
    withArguments: { 'user-789' }.
bookings inspect.

```

If you need a safe text summary, add a server-side diagnostic selector that returns a small dictionary rather than inspecting the full object graph.

```

OkzBookingRepository class >> debugSummaryForBookingId: aBookingId
  | booking |
  booking := self bookingWithId: aBookingId.
  booking ifNil: [
    ^ Dictionary new
      at: #found put: false;
      at: #bookingId put: aBookingId;
      yourself ].
  ^ Dictionary new
    at: #found put: true;
    at: #bookingId put: booking id;
    at: #eventId put: booking eventId;

```

```

    at: #userId put: booking userId;
    at: #status put: booking status;
    at: #paymentRecordId put: booking paymentRecordId;
    yourself

```

Then call it through the bridge:

```

| repository summary |
repository := GbsSessionParameters currentSession bridgeRoot
    at: #OkzBookingRepository.
summary := repository
    perform: #debugSummaryForBookingId:
        withArguments: { 'booking-123' }.
summary inspect.

```

This pattern is safer for production-like data because the diagnostic result is small, explicit, and non-mutating.

Fetch GemStone Transcript Output

For short-lived debugging, server-side code can write to the GemStone Transcript.

```

OkzPaymentRecordRepository class >> checkoutBookingId: aBookingId
paymentId: aPaymentId
    Transcript
        show: 'checkoutBookingId='; show: aBookingId;
        show: ' paymentId='; show: aPaymentId;
        cr.
    ...

```

From Pharo, fetch and clear the remote transcript:

```
GbsSessionParameters currentSession fetchTranscriptLogs
```

Use this for temporary development diagnostics only. For production, write structured operational records into a durable feature store or object log instead.

Abort After Failed Debug Runs

When debugging server-side code, assume the transaction may be dirty after an exception. Abort before retrying unless you intentionally want to commit the changes.

```

| session repository |
session := GbsSessionParameters currentSession.
[
    repository := session bridgeRoot at:
    #OkzPaymentRecordRepository.
    repository
        perform: #checkoutBookingId:paymentId:
            withArguments: { 'booking-123'. 'payment-456' }.
    session commitTransactionOrSignalConflict
] on: GbsError do: [ :error |

```

```
session abortTransaction.  
error inspect.  
error pass ]
```

For exploratory reads, aborting at the end is often the safest default:

```
[  
  "Inspect objects, run read-only diagnostics, reproduce the  
  error."  
] ensure: [  
  GbsSessionParameters currentSession abortTransaction  
]
```

Debug Argument Conversion Separately

Some failures are not server-side business bugs. They are bridge argument conversion bugs.

For example, remote message sends should use primitive values, common collections, dates/times, or existing GemStone proxy objects. Passing an arbitrary Pharo object can fail before the GemStone method even runs.

Bad:

```
repository  
  perform: #markPaidWithContribution:  
  withArguments: { anOkzPharoOnlyContributionObject }
```

Better:

```
repository  
  perform: #markPaidPaymentRecordId:externalPaymentId:  
  withArguments: {  
    anOkzPaymentRecord id.  
    'provider-payment-123'  
  }
```

or pass a dictionary made of primitive/common values:

```
repository  
  perform: #recordProviderPayload:  
  withArguments: {  
    (Dictionary new  
      at: #paymentRecordId put: anOkzPaymentRecord id;  
      at: #provider put: 'monzo';  
      at: #externalPaymentId put: 'provider-payment-123';  
      yourself)  
  }
```

A good rule is: cross the bridge with stable IDs and simple data. Let the GemStone resident repository look up the durable objects.

Debugging a Production Failure Safely

For production incidents, the workflow should be:

1. Capture the failing selector, arguments, user id, event id, booking id, payment id, and timestamp from logs or operational records.
2. Check the production health and GemStone availability dashboards.
3. Reproduce the selector against a staging restore or a copied extent if the bug is data-specific.
4. Use GbsRemoteDebugger in staging to inspect the server-side stack.
5. Patch the resident GemStone method or Pharo gateway call.
6. Run the focused regression test.
7. Run the preflight gate.
8. Deploy and verify with a concrete production smoke check.

The bridge is still central to this workflow, but not because you leave a live debugger attached to production. It is central because it gives you a faithful way to run the same remote message against GemStone in a controlled session.

Add Tests at the Server-Side Boundary

Every debugging session should ideally end with a test at the boundary where the bug happened.

For a gateway issue:

```
OkzGemStoneGatewayTest >>
testCheckoutBookingUsesResidentRepository
  | gateway result |
  gateway := OkzGemStoneGatewayForTest new.
  result := gateway
    checkoutBookingId: 'booking-123'
    paymentId: 'payment-456'.
  self assert: (result at: #status) equals: 'confirmed'
```

For a repository issue:

```
OkzRepositoryTest >>
testPaidCheckoutUpdatesBookingAndPaymentTogether
  | repository result booking payment |
  repository := OkzRepository new.
  result := repository
    checkoutBookingId: 'booking-123'
    paymentId: 'payment-456'.
  booking := result at: #booking.
  payment := result at: #paymentRecord.
  self assert: booking status equals: 'booked'.
  self assert: payment status equals: 'paid'
```

For a server-side method, the test should assert the behaviour of the selector, not the implementation detail that happened to fail.

The Practical Answer

Use the bridge as the interactive debugger in development:

- GbsWorkspace to run server-side snippets.
- GbsRemoteDebugger for GemStone exceptions and halts.
- Remote inspectors for returned GemStone objects and proxies.
- bridgeRoot to resolve resident repositories.
- perform:withArguments: to reproduce the exact server-side selector.
- commitTransactionOrSignalConflict and abortTransaction to control transactions explicitly.

Use production diagnostics for production:

- Structured operational records.
- Object logs.
- GemStone stone/gem logs.
- Staging restores.
- Focused replay scripts.
- Preflight gates and smoke checks.

That combination gives the best of both worlds: interactive Smalltalk debugging when developing the server-side code, and safe operational debugging when the live website is handling real users and payments.

Phase 3: Replace Fallbacks With Fail-Closed Behaviour

Fallbacks are useful during migration. They are dangerous after migration.

An old fallback path can make a system appear healthy while silently reading stale local state or executing retired bridge code. That is exactly what a single-store migration is meant to prevent.

The audit therefore pushed the system to fail closed:

- If a resident repository is unavailable, production should report an explicit unavailable state.
- It should not silently read old Pharo-side collections.
- It should not run retired script builders.
- Tests should enforce that fallback execution does not reappear.

This is stricter, but it is safer. A visible failure is better than a stale success in payments, refunds, registrations, or operational dashboards.

Phase 4: Collapse the Pharo Repository

The largest visible improvement came from deleting and splitting the old Pharo-side repository surface.

At one point, the audit showed the gateway and repository layers were still far too large. The gateway had grown instead of shrinking because both old and new paths coexisted. The repository still looked like a real in-memory store rather than a cache.

That was the real test of the migration.

Adding GemStone classes is useful. Removing the old parallel store is the payoff.

The final shape is much smaller:

- OkzGemStoneGateway is under the strict 500-line gate.
- OkzRepository is a small dispatch/cache surface.
- OkzBaseRepository has been reduced to a tiny compatibility shell.
- Legacy write-ahead-log and drift-detector machinery has been retired.
- Repository domain code is split into explicit helper domains rather than one giant object pretending to be a database.

The exact line counts matter less than the direction, but they are useful audit signals. Large migration projects often stop when the new path works. This one kept going until the old path was removed or fenced off.

That is the difference between “we added GemStone support” and “GemStone is now the source of truth.”

Phase 5: Make Feature Stores Durable

OkZulu has many operational feature stores: payment reconciliation records, email delivery history, webhook replay logs, backup evidence, migration evidence, sponsorship records, admin alerts, and more.

These are not all first-class domain objects, but many are still operationally important. Losing or duplicating them can affect support, compliance, reporting, or payment recovery.

The improved design classifies durable feature stores explicitly:

- Critical collections are mirrored through GemStone state-store records.
- Native dictionary stores are preserved as dictionaries where appropriate.
- Tests scan literal feature-store accessors and fail if a newly introduced store is not classified.
- Empty GemStone state is treated as authoritative empty, not ignored as “no data.”

That last point is subtle but important.

In a canonical server-side system, an empty remote collection can mean “there are no records.” It must not be treated the same as “the remote read failed.” The migration made that distinction explicit.

Phase 6: Prove Operations, Not Just Code

A database migration is not complete when tests pass. It is complete when the operating model is credible.

For OkZulu, that meant adding or tightening proof around:

- GemStone resident repository deployment.
- Gateway script-builder audits.
- Repository size gates.
- Backup producer wiring.
- Restore-drill runbooks.
- Performance baselines.
- Runtime evidence dashboards.
- Payment smoke checks.
- Cutover and post-cutover evidence.

The backup story was especially important. A backup that has never been restored is not a proven backup. The runbooks now distinguish between producing backups, fetching backups, and proving a restore drill.

That operational discipline is part of the architecture. It is not paperwork around the architecture.

What Improved

The migration improved the website in several concrete ways.

First, durability is clearer. Important business state now has a single canonical home in GemStone. Pharo renders and coordinates, but it does not pretend to be the durable database.

Second, failure modes are cleaner. Retired fallback paths no longer silently hide missing resident repositories. If the GemStone side is unavailable or incomplete, the system can report that directly.

Third, transactions are stronger. Server-side operations can update related objects and indexes together. That matters for checkout, refunds, tickets, sponsorships, and operational records.

Fourth, auditability improved. It is easier to inspect resident repository classes and remote selectors than to reason about generated GemStone source strings assembled in Pharo.

Fifth, the bridge became simpler. A small gateway that sends well-defined messages is easier to maintain than a large gateway that contains both script-building and object-dispatch logic.

Sixth, future feature work is safer. New durable feature stores must be classified. Repository size gates prevent a slow return to the old monolith. Tests catch fallback reintroduction.

A Concrete Example: Payments and Fundraising

OkZulu now has flows for tickets, sponsorships, standalone fundraisers, donor privacy, organiser notifications, receipts, refunds, and payment-provider fallbacks.

Those flows are exactly where server-side ownership matters.

When a sponsorship payment is marked paid, the system needs to update the payment record, update the contribution, queue donor receipts, notify the organiser, update fundraising analytics, and preserve enough history for support and reconciliation.

In the old model, this kind of workflow could easily become a chain of local updates plus remote mirrors. In the improved model, the durable pieces are classified and routed through GemStone-backed repositories and state stores.

The website still feels like a Seaside application. The improvement is underneath: fewer ambiguous writes, fewer hidden fallbacks, and clearer operational evidence.

Lessons From the Migration

The biggest lesson is that migration is not only construction. It is demolition.

It is not enough to build resident repositories. The old repository must stop being authoritative.

It is not enough to add a new bridge path. The old script builders must disappear.

It is not enough to create backups. Restores must be exercised.

It is not enough to pass application tests. Preflight gates must prevent architectural regression.

The second lesson is that line counts can be useful architectural evidence. They are not perfect, but they are hard to fake. If a gateway is supposed to become a thin dispatcher and it grows from 2,400 lines to 7,800 lines, the migration has probably stalled. If it drops under 500 lines and script-builder matches go to zero, the old path is actually being removed.

The third lesson is that fail-closed behaviour is a feature. During migration, fallbacks help maintain continuity. After migration, fallbacks can hide broken production state. The system should be explicit about unavailable canonical stores.

The fourth lesson is that Smalltalk-to-Smalltalk persistence is powerful when used fully. GemStone is not just a place to put rows. It can hold the server-side object model, indexes, and domain operations. That is the point of using it.

The Result

OkZulu moved from a Pharo-heavy website with GemStone persistence toward a GemStone server-side website with a Pharo Seaside presentation layer.

That is a better fit for the system it has become.

The website can keep the productivity of Pharo and Seaside while giving durable business workflows to GemStone. The bridge is smaller. The repository surface is thinner. Server-side classes own the data. Operational evidence is part of the release process.

The final architecture is not just cleaner. It is more honest.

Pharo is excellent at the web image, UI composition, and developer feedback loop. GemStone is excellent at durable objects, transactions, shared server-side state, and long-lived operational data.

The improvement came from letting each side do the job it is best at.